Predicting GitHub Actions failures before the run

A from-scratch decision tree on self-collected CI data

ACISE – Project 2

1 Problem

A CI run costs time and machines, and a developer waits for its result. If the outcome of a run could be predicted from what is known when the run is created, likely failures could be run first and likely passes could be given lower priority. This project predicts whether a GitHub Actions run will fail, using only information available before the run starts: the change, its author and the history of the workflow.

Two targets are used. **T1** is every run. **T2** keeps only the runs whose previous run in the same lineage (the same pull-request branch or pushed branch) passed, plus the first run of each lineage. T1 contains the runs that merely continue a known failure, which the previous outcome already predicts (the Parrot baseline). T2 removes them, so it asks for the harder and more useful prediction: a failure after a pass. **T2 is the main target**; T1 is shown for comparison.

The model is a CART decision tree written from scratch [1]. A tree is on the course list, needs no scaling, handles the imbalanced label with class weights, and explains every prediction as a short path of threshold tests. A random forest built from the same tree measures how much accuracy the explanation costs. Related work predicts build outcomes from history and change features [2, 3, 4] and uses such predictions to skip builds [5]; those studies use Travis CI data, whereas this project collects GitHub Actions data itself.

2 Data collection

Sources. 13 workflows of 11 open-source repositories were collected, each over every branch and pull request unless restricted to one branch (Table 1). The built tables span 2026-04-01 to 2026-11-01. The collection date C is the end of the last collected day; the frozen models are trained on runs created before C (2026-11-02, 00:00 UTC), and all later runs are new to them.

API. For each workflow the GitHub REST API is queried in windows of at most one month (halved down to one hour while a window holds 1 000 runs or more, the API's cap): `GET/repos/<repo>/actions/workflows/<id>/runs?created=<window>`, then `/actions/runs/<id>/attempts/1` for re-run runs and `/attempts/1/jobs` for failed first attempts. Every response is cached, so the collection can be resumed and repeated. Each repository is also cloned in full, including `refs/pull/*/head`; heads that are no longer referenced are fetched by SHA.

Rows and features. One row per run of a code-carrying event (push, pull request, merge queue). Features come from the Actions history (failure rates over time windows, flaky re-runs, CI load, queue times) and from git (churn, files and directories touched, test and source files, author experience, file failure history). History features of a run read only runs that *completed* before it was created, inside its own workflow, so no future information leaks in. Author identities are replaced by salted SHA-256 hashes; the salt never leaves the raw cache.

Labels. The label is the conclusion of the *first* attempt (a passing re-run does not hide a failure). Runs that were cancelled or ended otherwise are dropped. A failure whose failed jobs all match a per-workflow list of infrastructure jobs (or that has no failed job) is dropped as well: it did not come from the code. Of 213 113 collected runs, 150 535 are kept and 2 776 infrastructure failures are removed. T2 has 118 748 runs (16.6 % failed), T1 150 535 (24.7 % failed).

source	runs	kept	infra dropped	T2 rows	T2 fail %	T1 rows	T1 fail %
n8n-33322401	67381	48774	1055	35448	24.5	48774	32.6
n8n@master	7723	7186	5	5361	16.5	7186	26.1
ruff	29934	18356	335	15410	14.0	18356	18.3
uv	15910	9148	468	6905	23.8	9148	30.0
windmill-133951533	17919	13045	1	11957	4.6	13045	9.6
windmill-45664283	18913	14016	4	13343	1.7	14016	5.2
fd	487	456	3	394	17.8	456	20.6
gh-aw	14655	6537	164	3741	16.5	6537	48.2
brew	10006	8110	46	6970	11.5	8110	16.6
tokio	2536	1874	30	1345	29.1	1874	34.2
svelte	2363	1608	252	1292	18.6	1608	24.8
nushell	4014	2453	77	2036	15.6	2453	19.6
servo	21272	18972	336	14546	21.6	18972	27.5

Table 1: Sources. *runs*: collected (all events); *kept*: runs with a pass or fail label; *infra dropped*: infrastructure failures removed. Names carry the workflow id when a repository has two workflows.

3 Approach and implementation

Pooling. The 13 sources are pooled and ordered by creation time, as if they were one diverse workflow. History features are computed per source before pooling. The first 20 % of the pooled rows form the development segment, used for exploration and feature selection.

Feature selection. On the development segment only: near-constant and mostly missing columns are removed, every feature is ranked by relevance statistics (ANOVA F and Kendall τ for numeric features, χ^2 for binary ones, mutual information for all), the ranking is repeated on 20 block subsamples, features chosen in at least 60 % of them are kept, and of two features correlated above $\rho = 0.8$ only the more relevant stays. For T2, a feature that falls out of the top 20 in three or more CV training folds is then removed (drift filter). The frozen lists hold 18 (T2) and 20 (T1) features (Appendix, Table 8).

CART. Binary splits $x \leq t$ in raw units, Gini or entropy impurity, optional balanced class weights, minimal cost-complexity pruning. The grid covers depth 3–12, minimum leaf size 5–20, both criteria and both weightings. The pruning α and the decision threshold on $P(\text{fail})$ are chosen on the last 20 % of each training window, maximising MCC; one threshold is used for the whole pool. scikit-learn's tree is used only in the tests, as a reference: on the same inputs it gives the same predictions. The random forest bags 100 of our trees on bootstrap samples, with $\sqrt{p}$ features per node.

Evaluation. Expanding-window cross-validation with 5 folds after the development segment: each fold trains on all earlier runs and tests on the next block, so no model sees the future. The metric is the Matthews correlation coefficient (MCC) of the failure class. Baselines: *Parrot* (the lineage's previous outcome), *majority class*, and *source_rate*, which predicts failure when the run's source has a training failure rate above the threshold. A variant chooses one threshold per source.

Methods are compared with paired Wilcoxon tests and the Vargha–Delaney A_{12} effect size on (fold, source) cells with at least 10 test failures.

Frozen models. One CART and one forest per target are trained on all runs before C with the same protocol and saved with skops, next to a JSON sidecar that holds the features, hyperparameters, threshold, training medians (to impute new runs), source list and the file's SHA-256. Loading checks the hash and the allowed types. The frozen T2 CART has depth 10 and threshold 0.600 (94 999 training and 23 749 validation runs).

4 Validation

4.1 Cross-validation

method	T2 MCC	T2 recall	T1 MCC	T1 recall
cart	0.223 +/- 0.053	0.536	0.374 +/- 0.035	0.482
cart_source_thr	0.161 +/- 0.049	0.471	0.354 +/- 0.038	0.441
forest	0.247 +/- 0.060	0.562	0.405 +/- 0.046	0.511
forest_source_thr	0.191 +/- 0.034	0.353	0.397 +/- 0.041	0.496
source_rate	0.185 +/- 0.021	0.916	0.191 +/- 0.037	0.850
parrot	0.000 +/- 0.000	0.000	0.353 +/- 0.031	0.451
majority	0.000 +/- 0.000	0.000	0.000 +/- 0.000	0.000

Table 2: Test MCC on the pooled rows, mean ± std over the 5 folds, and mean recall of the failure class.

On T2 the CART reaches an MCC of 0.223 and the forest 0.247, against 0.185 for *source_rate*; Parrot and majority are 0, since every T2 run follows a pass (Table 2). On T1 the CART (0.374) is level with Parrot (0.353) and the forest is ahead (0.405). Per-source thresholds do not help on the pooled rows: they even out the flag rate across sources and give up the difference in base rates between them. Table 3 gives the paired tests on the (fold, source) cells.

target	comparison	cells	mean diff	p	A12
T2	cart vs cart_source_thr	62	-0.0031	0.474	0.479
T2	forest vs cart	62	+0.0183	0.200	0.535
T1	cart vs cart_source_thr	63	+0.0028	0.616	0.505
T1	forest vs cart	63	+0.0313	< 0.001	0.563
T1	cart vs parrot	63	-0.0008	0.843	0.517

Table 3: Paired Wilcoxon test and A_{12} on (fold, source) MCC cells. Cells of one fold share a model and a time window, so they are not independent.

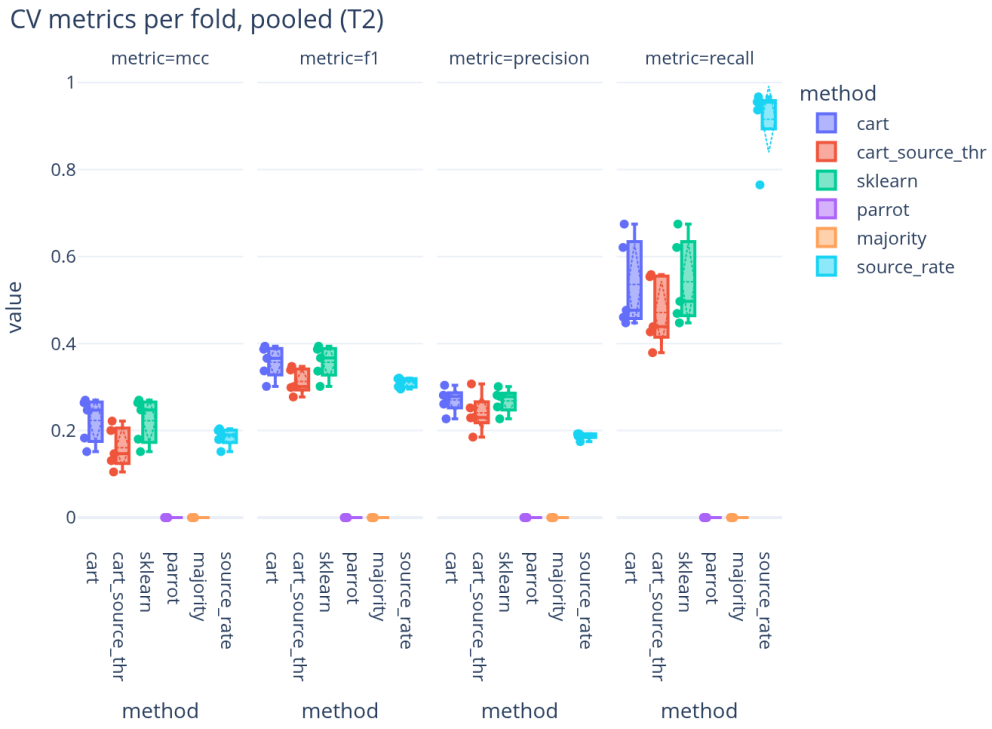


Figure 1: Test MCC per fold and method, T2.

4.2 Transfer between repositories (E1–E3)

E2 is the pooled model, scored on each source. E1 trains a CART on one source's own runs and feature list; E3 trains the pooled model without the source's repository and tests on that source, so the repository is unseen. All three use the same test rows. Over 62 cells, the pooled model is level with the per-source models (E2 − E1: -0.0001, $p = 0.964$, $A_{12} = 0.483$). Leaving the repository out costs accuracy (E3 − E1: -0.0239, $p = 0.009$, $A_{12} = 0.402$), see Table 4.

source	E2 pooled	E1 own	E3 left out
brew	0.193	0.135	0.126
fd	0.198	0.059	0.218
gh-aw	-0.004	0.019	-0.065
n8n-33322401	0.179	0.202	0.026
n8n@master	0.029	0.014	0.009
nushell	0.110	0.168	0.088
ruff	0.163	0.152	0.128
servo	0.131	0.141	0.057
svelte	0.166	0.155	0.221
tokio	0.230	0.128	0.236
uv	0.124	0.126	0.108
windmill-133951533	0.035	0.069	0.098
windmill-45664283	0.019	0.131	0.015

Table 4: T2 test MCC per source, mean over the 5 folds: pooled model (E2), the source's own model (E1), and the pooled model trained without the source's repository (E3).

4.3 Re-test on runs after the collection date

The frozen models were applied, unchanged, to every run created after C (8 770 T2 and 10 843 T1 runs). Their features read the stored history plus earlier new runs only. On T2 the CART scores 0.232 and the forest 0.260; on T1, 0.402 and 0.421 (Table 5). These are at or above the CV means, so the runs after C show no decay yet. Per source (Table 6), the scores spread widely and the small sources have few failures, so a single run moves their MCC a lot.

target	model	runs	failures	MCC	precision	recall	Parrot MCC
T2	cart	8770	1288	0.232	0.308	0.431	0.000
T2	forest	8770	1288	0.260	0.352	0.403	0.000
T1	cart	10843	2393	0.402	0.647	0.389	0.366
T1	forest	10843	2393	0.421	0.647	0.421	0.366

Table 5: Re-test of the frozen models on runs created after C, pooled.

source	runs	failures	CART MCC	forest MCC
brew	531	57	0.229	0.294
fd	23	2	0.265	0.339
gh-aw	103	24	0.127	-0.219
n8n-33322401	2666	508	0.227	0.255
n8n@master	436	46	0.011	0.033
nushell	205	32	0.093	0.288
ruff	1244	148	0.140	0.101
servo	1088	264	0.081	0.196
svelte	170	28	0.196	0.242
tokio	109	33	0.448	0.315
uv	321	92	0.193	0.123
windmill-133951533	913	36	0.187	0.366
windmill-45664283	961	18	0.349	0.331

Table 6: T2 re-test per source.

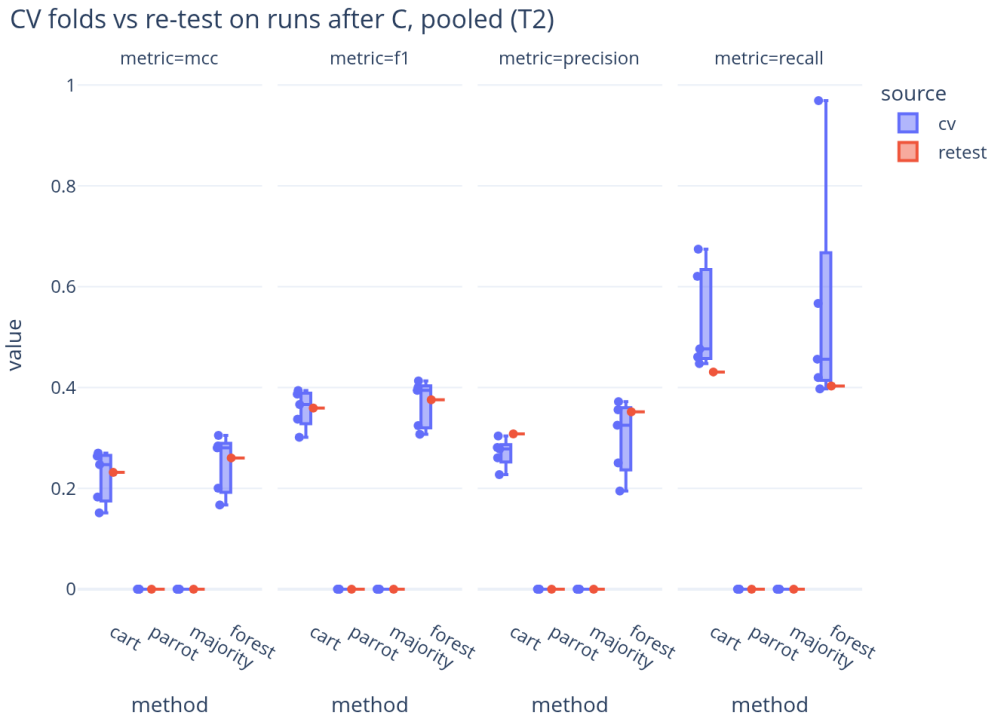


Figure 2: CV folds (boxes) and the re-test after C (points), T2, pooled.

4.4 Explained predictions and a live run

`predict-run` fetches one run, builds its row from its own source's history, and prints the CART's decision path in raw feature units. The validation notebook shows one true positive, true negative, false positive and false negative for each of the sources with the most re-test failures, and predicts a run that is still in progress: the actual use case, where the outcome is not yet known.

4.5 Sandbox (E4)

The sandbox is a branch of this project's own repository with a small Python package, real tests and one workflow. The models have never seen it. After 30 unbroken warm-up changes, 48 scored changes were pushed (38 % broken), every change shape both broken and unbroken. A broken change holds either a code defect (a mutation kept only if a test catches it) or a CI break in the workflow file. Every run was predicted while it was running, a median of 42 s before it ended; 0 runs could not be scored.

On the 73 scored runs (28 failed) the T2 CART predicted no failure: its highest $P(\text{fail})$ was 0.420, below the threshold 0.600. Its MCC is therefore 0.000, as for Parrot and majority (Table 7), and the mean $P(\text{fail})$ is almost the same for unbroken, code-broken and CI-broken changes. The decision paths show why. The tree splits first on the event, and every pull-request run reached the same leaf. The generated defects (a flipped comparison, an off-by-one, a broken workflow step) change what the code does, not its size or place, while the features describe the size and place of a change and the history around it; a broken change and an unbroken one of the same shape look alike to them. Each change was pushed on its own branch, so every run opened its lineage (100 %): Parrot had no previous outcome to repeat, and the lineage failure rates were filled with the training medians. The workflow-wide failure rates did rise with the broken changes and moved later push runs to riskier leaves, but not past the threshold. The sandbox thus shows that the pipeline works end to end on a live repository, and that the model cannot tell a semantic defect

from a harmless edit of the same size.

method	n_test	n_fail	mcc	precision	recall
cart	73	28	0.0	0.0	0.0
parrot	73	28	0.0	0.0	0.0
majority	73	28	0.0	0.0	0.0

breakage	runs	failed	predicted_fail	mean_p_fail
ci	8	8	0	0.245
code	20	20	0	0.255
none	45	0	0	0.245

Table 7: Sandbox runs: scores of the T2 CART, Parrot and majority, and the runs per breakage kind.

5 Limitations

- The CART grid stops at depth 12; deeper trees were not explored.
- Several T2 sources stay near an MCC of 0 (Table 4): the pre-run signal does not carry over to every workflow.
- The (fold, source) cells of one fold share a model and a time window, so the paired tests treat dependent samples as independent.
- The event features (push or pull request) differ in mix between sources, so they may partly stand in for the source rather than describe the change.
- Failures caused by flaky tests or the CI environment cannot be predicted from pre-run data; the infrastructure filter removes only the ones it recognises.
- The features describe the size, place and history of a change, not its content: the sandbox's generated defects were invisible to them. Each sandbox change also had its own branch, so the lineage features were always missing.

6 Future work

Per-repository fine-tuning on top of the pooled model; job-level prediction, so that only the jobs likely to fail run first; features from the build logs of earlier failures, as in BuildFast [3]; and a cost model that weighs a missed failure against a delayed pass, to choose the threshold for a concrete skipping or ordering policy.

References

[1] Leo Breiman et al. *Classification and Regression Trees*. Wadsworth, 1984.

[2] Foyzul Hassan and Xiaoyin Wang. "Change-Aware Build Prediction Model for Stall Avoidance in Continuous Integration". In: *Proceedings of the ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 2017, pp. 157–162.

[3] Bihuan Chen et al. "BuildFast: History-Aware Build Outcome Prediction for Fast Feedback and Reduced Cost in Continuous Integration". In: *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2020, pp. 42–53. DOI: 10.1145/3324884.3416616.

[4] Islem Saidani et al. “Predicting continuous integration build failures using evolutionary search”. In: *Information and Software Technology* 128 (2020), p. 106392.

[5] Xianhao Jin and Francisco Servant. “A Cost-efficient Approach to Building in Continuous Integration”. In: *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 2020, pp. 13–25. DOI: 10.1145/3377811.3380437.

A Frozen feature lists

index	T2	T1
1	fail_rate_24h	fail_rate_last_5
2	fail_rate_7d	fail_rate_12h
3	author_prior_fail_rate	fail_streak
4	fail_rate_6h	author_prior_fail_rate
5	file_fail_prob_max	fail_rate_3h
6	flaky_rate_7d	file_fail_prob_max
7	fail_rate_1h	fail_rate_1h
8	event_push	secs_since_last_fail
9	days_since_workflow_change	days_since_workflow_change
10	event_pull_request	runs_since_last_fail
11	author_prior_commits	flaky_rate_7d
12	ci_load_24h	file_age_mean
13	concurrent_runs	commits_since_green
14	lineage_prior_runs	src_churn
15	src_churn	event_pull_request
16	dir_entropy	dir_entropy
17	fail_rate_last_50	file_prior_changes
18	churn	ci_load_24h
19		days_since_last_tag
20		additions

Table 8: Frozen feature lists, in relevance order.